AeroScale — Final Jury Defense & Viva Guide

DISTRIBUTED AIRLINE SEAT INVENTORY • 60M DATASET • CITUS 14.2 & POSTGRESQL 18.4

OFFICIAL JURY DEFENSE

CQRS • Two-Phase Saga • Zero-Oversell Proof

1. EXACT JURY DEMONSTRATION: WHAT TO OPEN & RUN (STEP-BY-STEP)

PRESENTATION ORDER

1 **Open Screen 1: Real-Time Control Room & Concurrency Lab (/admin/lab)**

What to show: Open `http://localhost:5173/admin/lab`. Highlight the live Citus topology badges: **Write Coordinator (Port 55432)**, **CQRS Read Model (Port 5433)**, and **2 Worker Nodes**. Show the **60,000,006 Dataset Scale** and **22,064,356 Active Shard Counters**.

2 **Execute the "500-Concurrency Zero-Oversell Burst Test" Live Before the Jury**

The Killer Demo: In the Control Room, click **"Run 500-Concurrency Test"**. Explain: *"500 parallel shoppers are simultaneously attacking the exact same flight cabin with only 2 remaining seats."* Show the verified result: **Exactly 2 bookings succeed**, **498 are cleanly rejected with 409 Conflict**, remaining seats = **0**. 0 duplicate bookings, 0 oversell, 100% ACID safety.

3 **Open Screen 2: 60M Flight Dataset Search & Interactive Seat Map (/flights/search)**

What to show: Show that searching with no filters queries across **60,000,006 scheduled flights** in sub-40ms. Demonstrate page navigation (Page 1 of 6,000,001 with +100 jump). Click a flight to open the **Interactive Boeing 777 Seat Map**. Show that guests can browse freely, but clicking "Book Seat" triggers authentication with full redirect preservation.

4 **Open Terminal / Docker / pgAdmin: Raw Database Proof (If Jury Asks)**

Run `docker ps` to show Citus Coordinator, Workers, CQRS replica, and Redis. In psql/DBeaver (Port 55432):

`SELECT * FROM citus_tables;` shows distributed tables; `SELECT * FROM citus_shards;` shows 51 active shards.

2. WHY WE USED THESE SPECIFIC TECHNOLOGIES (PLAIN-ENGLISH DEFENSE)

ARCHITECTURE RATIONALE

WHY CITUS? **Horizontal Distributed Sharding**

Problem: Standard PostgreSQL chokes on single-node disk I/O, table locking, and memory overhead when managing 60M+ flight records and high-frequency concurrent seat hold transactions.
Why Citus: Transforms PostgreSQL into a distributed database. Data is partitioned across worker nodes using `flight_id` as shard key. Scaling CPU cores and disk IOPS horizontally without rewriting SQL queries.

WHY CQRS? **Port 5433 Search Read Replica**

Problem: In airlines, 99.8% of user operations are flight searches; less than 0.2% are bookings. Heavy search queries lock tables and starve critical checkout transactions.
Why CQRS: Complete Command-Query Separation. All search traffic hits a dedicated denormalized Read Replica (Port 5433) in < 40ms. The Write Coordinator (Port 55432) remains 100% dedicated to atomic bookings.

ROW-LEVEL CAS LOCKS **Zero Oversell**

Uses conditional SQL predicates:
`available_seats = available_seats - 1 WHERE available_seats >= 1`. Relies on Postgres MVCC row locks rather than slow Redis locks.

TRANSACTIONAL OUTBOX **Saga Integrity**

Seat reservation and outbox event are written inside the *same atomic ACID transaction*. Even if the server dies mid-flight, events are never lost and sagas auto-reconcile.

REDIS CACHE **Sub-2ms Speed**

Caches airport metadata, user JWT sessions, and hot flight searches. Relieves database from repeated queries while Citus handles financial-grade inventory balance.

Jury Elevator Pitch: *"AeroScale solves the billion-dollar airline problem: how to query 60 Million flights instantly without letting heavy search traffic interfere with the atomic row locks required to guarantee ZERO passenger overselling under extreme holiday booking spikes."*

3. HOW IT WORKS UNDER THE HOOD: THE 3 CORE ENGINEERING INVARIANTS

TECHNICAL DEPTH

INVARIANT 1 Citus Table Co-Location (Eliminating Distributed 2PC Overhead)

• Both `flights` and `flight_seats` tables are distributed with:

```
SELECT create_distributed_table('flight_seats', 'flight_id', colocate_with ⇒ 'flights');
```

• **Why this is crucial for the Jury:** Co-location guarantees that Shard 102001 of `flights` and Shard 102001 of `flight_seats` live on the *exact same physical worker node*!

• When a user books a flight, the query executes as a **single-node local ACID transaction** on that worker. It avoids expensive distributed network locks and cross-node Two-Phase Commit (2PC), giving enterprise throughput (> 2,500 bookings/sec).

INVARIANT 2 Mathematical Anti-Oversell Conditional Decrement (Compare-And-Swap)

• Instead of a naive `SELECT available_seats` followed by application-level `UPDATE` (which creates race conditions), AeroScale uses atomic database-level predicate locking:

```
UPDATE flight_seats SET available_seats = available_seats - 1 WHERE flight_id = $1 AND cabin_class = $2 AND
available_seats ⩾ 1 RETURNING available_seats;
```

• **Concurrency Invariant:** PostgreSQL MVCC serializes row updates on the unique `(flight_id, cabin_class)` tuple. If counter is 0, the WHERE clause evaluates to false; `rowCount = 0`. The transaction rolls back immediately and returns **409 Conflict**. It is mathematically impossible to oversell even with 1,000 racing threads.

INVARIANT 3 The 300-Second Hold & Two-Phase Saga Auto-Reconciliation

• When a passenger selects a seat, inventory is decremented and marked `status = 'HELD'` with `expires_at = NOW() + INTERVAL '5 minutes'`.

• An event is atomically recorded in `outbox_events`. A background Saga Reconciler polls for uncommitted holds. If the passenger abandons payment or the browser disconnects after 300s, the Saga triggers a **compensating transaction**: restoring `available_seats = available_seats + 1`, marking booking `'EXPIRED'`, and returning the seat to the open market.

4. ARCHITECTURAL COMPARISON: TRADITIONAL SYSTEM VS. AEROSCALE

SYSTEM COMPARISON

Engineering Dimension	Traditional Monolithic Architecture	AeroScale Distributed Engine
Data Volume & Scale	Single database instance degrades past 5M-10M rows due to B-tree depth and memory swapping.	**60,000,006 rows** horizontally sharded over 51 Citus partition shards; scales linearly with new worker nodes.
Concurrency & Oversell	Pessimistic table locks or application-side Redis locks. Prone to race conditions and lock convoys.	**Strict conditional row-level predicate updates** (CAS) enforced directly by PostgreSQL storage engine. 0% oversell.
Search vs Booking Contention	Heavy passenger flight searches share disk/CPU with checkout transactions, causing timeouts.	**CQRS Architecture:** Search queries execute against Port 5433 read replica (< 40ms); Port 55432 coordinator handles writes.
Partial Failure Resilience	Direct synchronous microservice calls leave dangling holds if network drops mid-booking.	**Transactional Outbox + Saga Orchestrator:** Guaranteed at-least-once delivery with 300s compensating reconcilers.
Worker Daemon Auth	Workers cannot coordinate maintenance tasks when passwords missing across internal Docker containers.	**Hardened Trust / Pgpass Matrix:** Worker1 and Worker2 communicate seamlessly for shard rebalancing.

Shard Distribution Key Choice

`flight_id` distributes flights evenly across shards using a hash algorithm. Flights departing on the same date are spread evenly across all worker disks.

Indexed Cursor Pagination

Instead of `OFFSET 50000000` (which sequentially scans 50M rows), the CQRS engine uses indexed covering indexes on `(departure_time, flight_id)` for instant jumps.

AeroScale — Advanced Viva Defense Questions & Cheat-Sheet

HIGH-SCORING THEORETICAL & PRAGMATIC ANSWERS • EMERGENCY CLUSTER REFERENCE

VIVA DEFENSE MASTER

CAP Theorem • MVCC • Deadlocks • 2PC

5. ADVANCED VIVA QUESTIONS & HIGH-SCORING DEFENSE ANSWERS

EXAMINER Q&A

Q1 Why Citus sharding instead of standard Postgres primary-replica setup?

Answer: Standard read replicas only scale *read operations*; all *writes* still hit one single master disk and CPU, creating an absolute bottleneck. Citus shards both tables and indexes across worker nodes, providing horizontal scaling for **both reads and concurrent writes** while allowing a 60M dataset to exceed single-machine RAM limits.

Q2 How does AeroScale adhere to the CAP Theorem?

Answer: We make a deliberate split based on business domain: For **Seat Inventory & Bookings**, we choose **CP (Consistency & Partition Tolerance)** — if network partitions occur, we fail fast rather than risk overselling. For **Flight Browsing/Search**, we choose **AP (Availability & Partition Tolerance)** via the CQRS replica where eventual consistency (lag < 50ms) is completely acceptable.

Q3 What prevents distributed deadlocks during simultaneous multi-seat bookings?

Answer: Two mechanisms: First, **shard co-location** ensures all seat counters for a given flight live on the exact same worker, meaning locks are resolved by Postgres's local deadlock detector. Second, our application enforces a **deterministic lock acquisition order** (sorting seat IDs in ascending order before executing updates), mathematically preventing circular lock wait chains.

Q4 What transaction isolation level is used and why not Serializable?

Answer: We use **Read Committed with Atomic Conditional Updates**. `SERIALIZABLE` isolation aborts high numbers of transactions with serialization failure errors (40001) under heavy concurrency, requiring complex retry loops. Our atomic conditional decrement achieves 100% mathematical safety directly at the storage engine level with zero retry penalty.

Q5 What happens if the server crashes right after taking the passenger's money?

Answer: The **Transactional Outbox pattern** guarantees safety: the booking record and outbox event are committed atomically in the same database transaction. The background Saga worker replays unhandled events with idempotent keys, ensuring notifications, ticket emission, and payment capture complete reliably with zero orphan bookings.

Q6 Why not store inventory counters in Redis using DECR instead of Citus?

Answer: Redis `DECR` is in-memory and fast, but Redis is not fully ACID. During an unexpected power loss or failover, asynchronous replication (RDB/AOF) can lose state, resulting in catastrophic overselling. In AeroScale, Citus acts as the single source of truth for financial-grade inventory, while Redis is used strictly for read caching.

Q7 What caused the Citus maintenance daemon error and how was it solved?

Answer: The maintenance daemon on `worker2` was failing to connect to `worker1:5432` with `fe_sendauth: no password supplied`. Citus workers require bidirectional internal communication for maintenance tasks. We resolved this by creating trust rules on the internal Docker network, allowing workers to authenticate transparently without exposing ports externally.

6. JURY CHEAT-SHEET: CLUSTER TOPOLOGY, PORTS & VERIFICATION COMMANDS

EMERGENCY CLI

Component	Host Port	Internal Role	Verification Command / SQL
Frontend Portal	`5173`	React Vite SPA (Search & Lab)	Browse to `http://localhost:5173/admin/lab`
Backend API	`3001`	Express CQRS & Saga Engine	`curl http://localhost:3001/api/health`
Citus Coordinator	`55432`	Distributed Write Master	`psql -h localhost -p 55432 -U airline -c "SELECT * FROM citus_tables;"`
CQRS Read Replica	`5433`	Denormalized 60M Search Replica	`psql -h localhost -p 5433 -U airline -c "SELECT count(*) FROM search_flights;"`
Citus Worker 1 & 2	`5434` / `5435`	Partition Shard Storage Nodes	`docker logs airline-citus-worker1 --tail 20`
Redis Cache	`6379`	Session & Search Projection Cache	`docker exec -it airline-redis redis-cli ping` (PONG)

Winning Defense Closing Statement: *"AeroScale is not a toy CRUD application; it is an industrial distributed systems architecture built to demonstrate that ACID safety, zero-oversell mathematical guarantees, and horizontal scalability across 60 Million records can be achieved simultaneously using modern PostgreSQL Citus sharding and CQRS."*